

Technik Programista

Zaawansowane aplikacje webowe

Flask – start

Środowisko, Hello World, pierwsze trasy

venv, pip, app.py, @app.route, serwer deweloperski, HTTP w praktyce

Klasa 4 | rok szkolny 2026/2027 | System: Windows

Spis treści

Spis treści

Spis treści.....	2
1. Czym jest Flask.....	3
2. Przygotowanie projektu i środowiska	3
2.1 Sprawdź Pythona.....	3
2.2 Folder projektu i venv	3
2.3 Instalacja Flaska	4
3. Hello World.....	4
4. Co robi każda linia.....	5
4.1 Gdzie tu jest MVC?.....	5
5. Kilka tras	6
5.1 Zwracanie czegoś więcej niż string.....	6
6. HTTP w praktyce: co widzi przeglądarka	6
7. Najczęstsze problemy	7
8. Do repozytorium	8
9. Zadania	9
Zadanie 1: Środowisko	9
Zadanie 2: Hello World	9
Zadanie 3: Cztery trasy.....	9
Zadanie 4: Statusy.....	9
Zadanie 5: JSON	9
Zadanie 6: Zepsuj i napraw	9
Zadanie 7: Repo	9

1. Czym jest Flask

Flask to mikroframework webowy dla Pythona. „Mikro” nie znaczy, że nadaje się tylko do małych rzeczy – znaczy, że **rdzeń jest mały**: routing (który URL → która funkcja), obsługa żądań i odpowiedzi, szablony. Bazę danych, logowanie, formularze dokładasz sam, biblioteką, którą wybierzesz.

	Flask	Django
Typ	mikroframework	framework „z bateriami”
Struktura projektu	dowolna, nawet jeden plik	narzucona (apps, models, views, urls)
Baza danych	dokładasz (SQLAlchemy)	wbudowany ORM
Panel admina	brak	wbudowany
Kiedy	API, małe serwisy, nauka od środka	duże aplikacje, szybki start CRUD

💡 *Uczymy się Flaska pierwszego właśnie dlatego, że nic nie ukrywa. Zobaczysz gołe żądanie i odpowiedź. Potem Django będzie „aha, to robi to samo, tylko za mnie”.*

2. Przygotowanie projektu i środowiska

2.1 Sprawdź Pythona

```
python --version
```

Potrzebujesz 3.10 lub nowszego. Jeśli komenda nie działa, spróbuj `py --version`.

2.2 Folder projektu i venv

Środowisko wirtualne (venv) to osobny, izolowany Python dla tego jednego projektu. Biblioteki zainstalowane w venv nie mieszają się z innymi projektami ani z systemem.

```
mkdir flask-start
cd flask-start
python -m venv venv
venv\Scripts\activate
```

Po aktywacji prompt zaczyna się od `(venv)`. **Od teraz wszystko robisz z aktywnym venv.**

⚠️ **PowerShell może odmówić: „running scripts is disabled”. Wykonaj raz: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` – i spróbuj ponownie. Alternatywnie użyj `venv\Scripts\activate.bat` w cmd.**

2.3 Instalacja Flaska

```
pip install flask
pip freeze > requirements.txt
```

`requirements.txt` to lista bibliotek projektu. Ktoś, kto sklonuje Twoje repo, robi `pip install -r requirements.txt` i ma to samo środowisko. **Ten plik idzie do Gita; folder ``venv/`` nie.**

3. Hello World

Utwórz plik `app.py`:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def index():
    return "Hello World!"

if __name__ == "__main__":
    app.run(debug=True)
```

Uruchom:

```
python app.py
```

W terminalu zobaczysz coś takiego:

```
* Running on http://127.0.0.1:5000
* Debugger is active!
```

Otwórz w przeglądarce **`http://127.0.0.1:5000`** (albo `http://localhost:5000`). Widzisz „Hello World!”? Masz działający serwer WWW.

💡 *Serwer działa, dopóki nie zatrzymasz go w terminalu: `Ctrl + C`. Z `debug=True` po każdej zmianie w `app.py` serwer sam się przestawia – wystarczy odświeżyć przeglądarkę.*

4. Co robi każda linia

Linia	Co robi
<code>from flask import Flask</code>	importujemy klasę Flask z biblioteki
<code>app = Flask(__name__)</code>	tworzymy aplikację. <code>__name__</code> mówi Flaskowi, gdzie szukać plików (szablonów, static)
<code>@app.route("/")</code>	dekorator : „gdy ktoś wejdzie na adres <code>/</code> , uruchom funkcję poniżej”
<code>def index():</code>	zwykła funkcja Pythona. Nazwa dowolna, ale ma być unikalna
<code>return "Hello World!"</code>	to, co funkcja zwraca, przeglądarka dostaje jako odpowiedź
<code>if __name__ == "__main__":</code>	wykonaj tylko, gdy plik uruchomiono bezpośrednio (<code>python app.py</code>), nie przy imporcie
<code>app.run(debug=True)</code>	start serwera deweloperskiego; <code>debug = True</code> = auto-przeładowanie i czytelne błędy

4.1 Gdzie tu jest MVC?

Funkcja `index()` to **kontroler**: odbiera żądanie i zwraca odpowiedź. Widoku nie ma (zwracamy goły tekst), modelu nie ma (nie ma danych). Na razie mamy samo C. To normalne – dobudujemy resztę w kolejnych materiałach.

⚠ debug=True tylko na własnym komputerze. Nigdy na serwerze dostępnym z internetu – debugger pozwala wykonać dowolny kod Pythona z przeglądarki.

5. Kilka tras

Trasa (route) to para: adres URL → funkcja. Dodaj do `app.py` kolejne:

```
@app.route("/")
def index():
    return "Strona główna"

@app.route("/o-nas")
def o_nas():
    return "Jesteśmy klasą 4 Technik Programista. Robimy sklep."

@app.route("/kontakt")
def kontakt():
    return "Napisz: sklep@example.com"
```

Sprawdź: `http://localhost:5000/o-nas` oraz `/kontakt`. A teraz wejdź na `/cennik` – dostaniesz **404 Not Found**, bo takiej trasy nie ma.

5.1 Zwracanie czegoś więcej niż string

Funkcja może zwrócić też kod statusu HTTP:

```
@app.route("/tajne")
def tajne():
    return "Brak dostępu", 403
```

Albo słownik – Flask sam zamieni go na JSON (tak będą wyglądały odpowiedzi API dla Reacta i Androida):

```
@app.route("/api/status")
def status():
    return {"ok": True, "wersja": "0.1"}
```

6. HTTP w praktyce: co widzi przeglądarka

Otwórz narzędzia deweloperskie (F12) → zakładka **Network** → odśwież stronę. Kliknij w żądanie do `localhost`. Zobaczysz:

- **Request URL** – co przeglądarka poprosiła
- **Request Method** – `GET` (pobierz). Później poznasz `POST` (wyślij dane)
- **Status Code** – `200 OK`, `404 Not Found`, `403 Forbidden`, `500 Internal Server Error`
- **Response** – to, co zwróciła Twoja funkcja

W terminalu, gdzie działa Flask, każde żądanie zostawia linię w logu:

```
127.0.0.1 - - [02/Sep/2026 10:15:32] "GET /o-nas HTTP/1.1" 200 -
```

💡 *To są dwa końce tej samej rozmowy: przeglądarka pyta, Flask odpowiada. Wszystko, co zrobimy przez cały rok – Django, React, Android, API – to warianty tej rozmowy.*

7. Najczęstsze problemy

Objaw	Przyczyna i rozwiązanie
<code>ModuleNotFoundError: No module named 'flask'</code>	venv nie jest aktywny. Zobacz, czy prompt ma <code>(venv)</code> . Aktywuj: <code>venv\Scripts\activate</code>
<code>Address already in use</code>	poprzedni serwer nadal działa. Zamknij tamten terminal (Ctrl + C) albo uruchom na innym porcie: <code>app.run(debug=True, port=5001)</code>
404 na stronie, która „na pewno jest”	literówka w <code>@app.route</code> albo w adresie. Trasa <code>/o-nas</code> $\neq$ <code>/o_nas</code> $\neq$ <code>/o-nas/</code>
<code>AssertionError: View function mapping is overwriting</code>	dwie funkcje o tej samej nazwie. Nazwy funkcji muszą być unikalne
Zmiana w kodzie nie działa	brak <code>debug=True</code> albo błąd składni – spójrz w terminal, tam jest komunikat
Polskie znaki wyświetlają się krzywo	na razie zwracamy goły tekst; rozwiąże się z HTML i <code><meta charset></code> w następnym materiale

8. Do repozytorium

Plik **.gitignore**:

```
venv/  
__pycache__/  
*.pyc  
.env  
.idea/  
.vscode/
```

Plik **README.md**:

```
# flask-start  
  
Pierwsza aplikacja Flask - trasy tekstowe.  
  
## Uruchomienie  
python -m venv venv  
venv\Scripts\activate  
pip install -r requirements.txt  
python app.py  
  
## Autor  
Imię Nazwisko, 4 Technik Programista  
  
git init  
git add .  
git commit -m "Flask: hello world i trasy tekstowe"  
git branch -M main  
git remote add origin https://github.com/TWOJ_LOGIN/flask-start.git  
git push -u origin main
```

⚠ Sprawdź na GitHubie, że folder venv/ NIE został wysłany. Jeśli jest – .gitignore powstał za późno. git rm -r --cached venv, commit, push.

9. Zadania

Zadanie 1: Środowisko

Załącz projekt `flask-start` według rozdziału 2. Pokaż nauczycielowi (`venv`) w prompcie i zawartość `requirements.txt`.

Zadanie 2: Hello World

Uruchom aplikację z rozdziału 3. Zmień tekst na swoje imię. Nie restartuj serwera – odśwież przeglądarkę. Zdziałało? Dlaczego?

Zadanie 3: Cztery trasy

Dodaj trasy `/`, `/o-nas`, `/kontakt`, `/regulamin`. Każda zwraca inny tekst związany z Twoją domeną projektu (sklep, portal, rezerwacje, ogłoszenia).

Zadanie 4: Statusy

Dodaj trasę `/admin`, która zwraca tekst „Brak dostępu” i kod 403. Sprawdź kod statusu w zakładce Network (F12).

Zadanie 5: JSON

Dodaj trasę `/api/info`, która zwraca słownik z kluczami: `nazwa` (nazwa Twojego projektu), `autor`, `wersja`. Otwórz w przeglądarce – zobaczysz JSON.

Zadanie 6: Zepsuj i napraw

Celowo zrób literówkę w `return` (np. `retrun`). Uruchom. Przeczytaj komunikat błędu w terminalu i w przeglądarce (debugger). Napraw. Zapisz w README, co pokazał debugger.

Zadanie 7: Repo

`.gitignore`, `README.md`, commit, push. Sprawdź na GitHubie, że `venv/` nie ma.